

La généricité en Java

Fabrice.Kordon@lip6.fr



Paramétrer une classe en Java

2

Indiquer le paramètre générique formel entre < et >

- Utilisable tel que dans la classe
 - ▶ Substitution par réécriture lors de l'instanciation
- Convention : paramètres génériques = une seule lettre majuscule
 - ▶ E → éléments (utilisation intensive dans le framework Collections)
 - ▶ K → Clef
 - ▶ N → nombre
 - ▶ T → type
 - ▶ V → valeur



Important

Usage intensif de la généricité dans les API de Java (Collections)

Instantiation

- Association de paramètre génériques effectifs
 - ▶ Création d'une classe autonome
 - ▶ Ne fonctionne qu'avec les types identifiés
 - ▶ Attention, les paramètres effectifs doivent être des Classes

Exemple simple, le container

3

```
class Container<T> {  
    T valeur;  
  
    Container(T default) {  
        valeur = default;  
    }  
  
    public void affecter (T nouveau) {  
        valeur = nouveau;  
    }  
  
    public T lire () {  
        return valeur;  
    }  
}  
  
public class ExempleTemplates1 {  
  
    public static void main(String []args) {  
  
        // Instantiation avec la classe Integer  
        Container<Integer> cInt= new Container<Integer>(3);  
        // Instantiation avec la classe String  
        Container<String> cString = new Container<String>("Titi");  
  
        System.out.println("avant, " + cInt.lire() + " et " + cString.lire());  
        cInt.affecter(7);  
        cString.affecter("Rominet");  
        System.out.println("après, " + cInt.lire() + " et " + cString.lire());  
  
    }  
}
```


Observations

4

Exécution du programme

```
$ java ExempleTemplates1  
avant, 3 et Titi  
après, 7 et Rominet
```



Rappel

Pas d'instantiation avec
des types primitifs

Quid de l'utilisation de int au lieu de Integer?

```
$ javac ExempleTemplates1.java  
ExempleTemplates1.java:24: error: unexpected type  
    Container<int> c= new Container<int>(3);  
        ^  
required: reference  
found:    int  
ExempleTemplates1.java:24: error: unexpected type  
    Container<int> c= new Container<int>(3);  
        ^  
required: reference  
found:    int  
2 errors
```


Méthodes génériques

5

```
class Comparateur {  
    public static <K, V> boolean compare(Couple<K,V> e1, Couple<K,V> e2) {  
        return e1.getClef().equals(e2.getClef());  
    }  
}
```



La méthode equals

Définir une «comparaison d'égalité»
Doit être surchargée pour toutes les comparaisons souhaitées
Object possède une implémentation par défaut

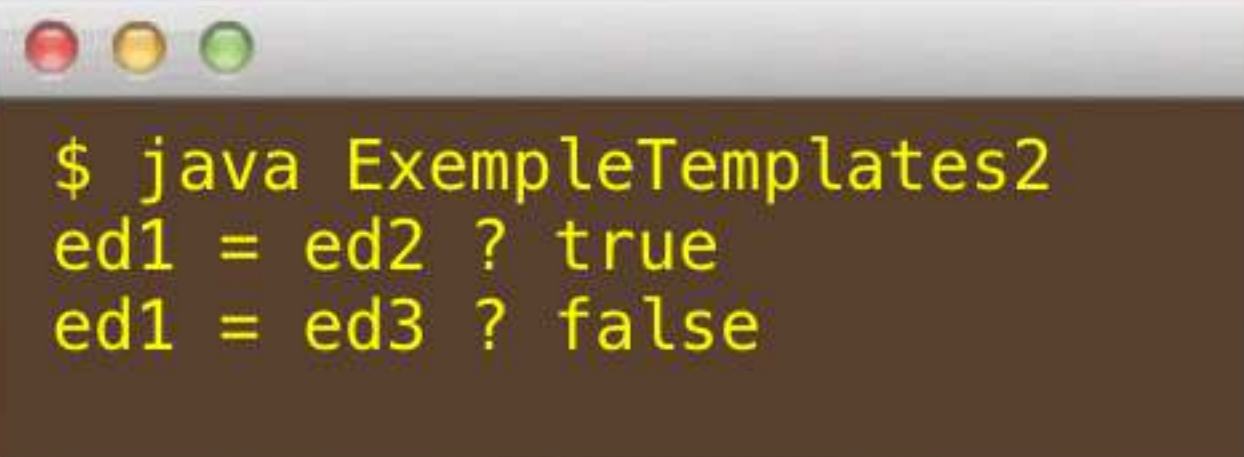
Méthodes génériques

5

```
class Compareteur {  
    public static <K, V> boolean compare(Couple<K,V> e1, Couple<K,V> e2) {  
        return e1.getClef().equals(e2.getClef());  
    }  
}
```

```
class Couple<K, V> {  
    private K clef;  
    private V valeur;  
  
    Couple(K k, V v) {  
        clef = k;  
        valeur = v;  
    }  
  
    public void setClef (K k) {  
        clef = k;  
    }  
  
    public void setValeur (V v) {  
        valeur = v;  
    }  
  
    public K getClef () {  
        return clef;  
    }  
  
    public V getValeur () {  
        return valeur;  
    }  
}
```

```
public class ExempleTemplates2 {  
    public static void main(String []args) {  
  
        Couple<Integer, String> ed1 = new Couple<Integer, String> (1, "Titi");  
        Couple<Integer, String> ed2 = new Couple<Integer, String> (1, "Grand-mère");  
        Couple<Integer, String> ed3 = new Couple<Integer, String> (2, "Rominet");  
  
        System.out.println ("ed1 = ed2 ? " + Compareteur.compare(ed1, ed2));  
        System.out.println ("ed1 = ed3 ? " + Compareteur.compare(ed1, ed3));  
  
    }  
}
```



```
$ java ExempleTemplates2  
ed1 = ed2 ? true  
ed1 = ed3 ? false
```


En guise de conclusion...

6

Mécanique extrêmement puissante

- Couplage avec le modèle Objet démontré et efficace
- Possibilité d'imbrication
- Possibilité de construire des itérateurs
- etc.

Très utile pour la factorisation de code

- À étudier pour la programmation «de haut vol»
 - ▶ Utile pour l'introspection
- Notion importante
 - ▶ Itérateurs (vu plus loin)



Mais...

On quitte le domaine de la
programmation concurrente!



One more consideration...



7

Implémentation de la généricité en java élégante

- Réécriture puis passage par la «mécanique usuelle»
- Aussi riche que la généricité d'Ada
- Mais écriture moins claire (avis personnel 🙄)
 - ▶ En Java : hypothèses de fonctionnement un peu dispersées dans le code
 - ▶ En Ada : hypothèses de fonctionnement bien posées (entre le mot clef generic et le début de «l'unité de programme» concernée)
- Mais aussi sûr dans les deux cas... une fois le code compilé
 - ▶ Contrôle de type
 - ▶ Contrôle des signatures
 - ▶ etc.



Suggestion pour java

- Lier vos classes à des interfaces (respect des hypothèses attendues)